



# PROG IV:

## Integração Full Stack + APIs Externas

---

Profª. Tainá Isabela



# Frontend e Backend

O desenvolvimento Full Stack fundamenta-se no princípio de projeto da separação de conceitos, dividindo o sistema em duas grandes camadas com funções distintas. O **frontend** é a **camada de apresentação** que roda no navegador do cliente, sendo responsável pela interface gráfica, interatividade e captura de eventos do usuário.

O **backend** atua como o motor lógico da aplicação no servidor, **gerenciando regras de negócio, persistência em bancos de dados e segurança de credenciais**. A comunicação entre essas duas camadas ocorre de maneira **assíncrona**, utilizando a rede de computadores para trafegar dados estruturados sem a necessidade de recarregar a página visual a cada nova requisição.

# O Papel das APIs na Computação em Nuvem

As APIs, ou **Interfaces de Programação de Aplicações**, funcionam como pontes de comunicação **padronizadas** que permitem a interação entre softwares de desenvolvedores diferentes. No ecossistema atual, **o consumo de APIs externas possibilita que uma aplicação local utilize serviços complexos de terceiros**, como processamento de pagamentos, mapas geográficos ou previsões meteorológicas.

Em vez de construir um sistema de cobrança bancária do zero, o desenvolvedor conecta o seu backend a um serviço consolidado como o Stripe ou o PayPal. Essa abordagem reduz drasticamente o tempo de desenvolvimento e transfere a responsabilidade de manutenção de infraestruturas críticas para empresas especialistas.

# O Protocolo HTTP e os Métodos de Requisição

A comunicação na Web é regida pelo protocolo HTTP, que opera em um modelo clássico de cliente-servidor por meio de mensagens de requisição e resposta. Cada requisição enviada pelo frontend ou pelo backend deve explicitar um método que define a natureza da operação pretendida no servidor.

Os métodos mais comuns são o GET para buscar dados, o POST para criar novos registros, o PUT para atualizar informações existentes e o DELETE para remover recursos. Compreender a semântica desses métodos é essencial para construir sistemas que respeitam as convenções da internet e facilitam a comunicação entre equipes de desenvolvimento de software.

# REST e Restrições Estateless

O padrão REST é o modelo arquitetural mais adotado para a construção de APIs modernas, operando sobre as diretrizes nativas do protocolo HTTP. Uma das restrições fundamentais do REST é o **comportamento estável e sem estado**, conhecido tecnicamente pelo termo inglês **stateless**.

Isso significa que cada requisição feita ao servidor deve conter absolutamente todas as informações necessárias para ser compreendida e processada isoladamente. O servidor não armazena o contexto das requisições anteriores na memória de sessão, o que simplifica o design da infraestrutura, melhora o desempenho e permite o escalonamento horizontal do sistema.

# Formato JSON para Intercâmbio de Dados

O **JSON** tornou-se o formato padrão universal para a troca de informações entre o frontend, o backend e as APIs externas devido à sua simplicidade. Trata-se de uma formatação leve de texto que representa dados na forma de **pares de chave e valor**, além de arrays ordenados de elementos.

Por ser um formato baseado em texto plano, o JSON é facilmente lido por seres humanos e interpretado rapidamente por praticamente qualquer linguagem de programação moderna. Durante o fluxo de integração, o JavaScript no frontend e o framework no backend convertem estruturas de dados internas em texto JSON para transmissão pela rede.

# Conectando o Frontend ao Backend com a Fetch API

A conexão do frontend com o backend é realizada por meio de requisições **assíncronas** utilizando recursos nativos do navegador, como a Fetch API. Através de funções baseadas em **promessas**, o código JavaScript **dispara uma chamada para a URL do backend sem travar a navegação do usuário na tela**.

O navegador envia os cabeçalhos apropriados e aguarda a resposta do servidor em segundo plano enquanto a interface continua operando. Assim que o backend **responde com os dados em formato JSON**, a promessa é resolvida, permitindo que o frontend **atualize dinamicamente os elementos visuais do documento HTML**.

# O Mecanismo de Segurança do CORS

O **Compartilhamento de Recursos de Origem Cruzada**, conhecido pela sigla **CORS**, é um mecanismo de segurança essencial implementado pelos navegadores web para proteger os usuários. Por padrão, um script executado no frontend só pode requisitar recursos da mesma origem de onde a página foi carregada.

Se o frontend tentar acessar um backend hospedado em um domínio ou porta diferente, o navegador bloqueará a resposta a menos que o backend envie cabeçalhos HTTP específicos liberando o acesso. Configurar o CORS corretamente no servidor backend é um passo obrigatório para permitir a comunicação legítima com a interface do usuário.



# Consumo de APIs Externas no Lado do Servidor

Embora o frontend possa consumir serviços externos diretamente, a boa prática de arquitetura dita que requisições para APIs de terceiros devem ser centralizadas no backend. Essa abordagem traz vantagens cruciais de segurança, pois chaves de acesso privadas e senhas de API nunca devem ficar expostas no código do cliente.

O backend atua como um intermediário seguro: ele recebe o pedido do frontend, anexa as credenciais secretas guardadas no servidor, faz a chamada para o provedor externo e processa a resposta. Essa estratégia também permite centralizar o tratamento de erros e aplicar regras de cache.

# Gerenciamento de Variáveis de Ambiente

Para garantir a segurança e a portabilidade do software, chaves de API, senhas de bancos de dados e URLs de servidores externos **nunca devem ser inseridas** diretamente no código-fonte. Utiliza-se a metodologia de isolamento por meio de variáveis de ambiente, geralmente configuradas em arquivos ocultos com a extensão **.env**.

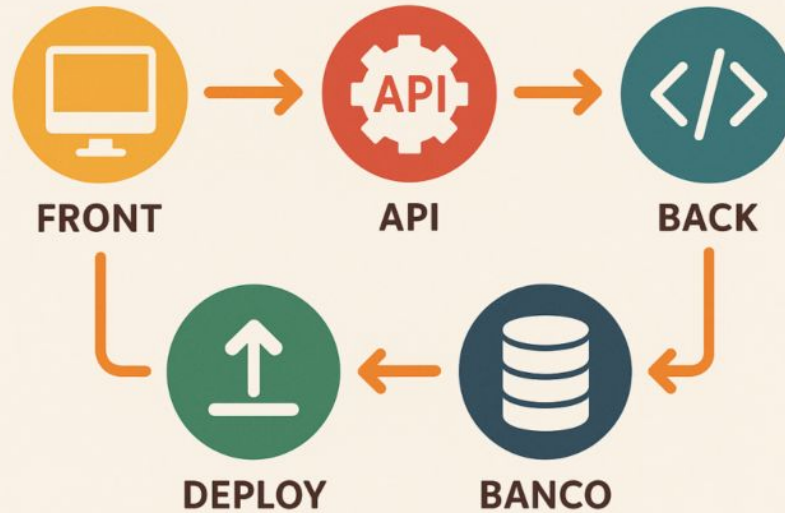
O código do backend lê esses valores diretamente do sistema operacional durante a execução da aplicação. Essa prática evita o vazamento accidental de credenciais confidenciais em repositórios públicos como o GitHub e facilita a troca de configurações entre os ambientes de teste e de produção.

# Fluxo de Dados Ponta a Ponta

Considere o exemplo prático de um aplicativo de comércio eletrônico que precisa calcular o valor do frete com base no CEP fornecido pelo cliente. O usuário digita o número no formulário do frontend, que dispara uma requisição POST contendo os dados para o backend próprio do sistema.

O servidor backend recebe o CEP, valida o formato do texto e efetua uma nova chamada HTTP para a API externa dos Correios ou de uma transportadora parceira. Ao receber o valor do frete calculado pelo serviço parceiro, o backend processa a informação e devolve um JSON limpo para o frontend, que exibe o preço final na tela do comprador.

# CICLO FULL STACK



# Integração com Gateways de Pagamento

Em sistemas de vendas reais, o processo de finalização de compra ilustra perfeitamente a necessidade de uma integração robusta em múltiplas camadas. O cliente insere os dados de pagamento na interface do frontend, que se comunica com o backend para registrar a intenção de compra no banco de dados local.

O backend aciona a API externa do banco ou operadora de cartão de crédito para processar a transação financeira de forma segura. Se o pagamento for autorizado externamente, o backend atualiza o status do pedido para "pago" e envia uma confirmação para o frontend exibir uma mensagem de sucesso, fechando o ciclo do serviço.

# A Importância dos Códigos de Status HTTP

Os códigos de status HTTP são números de três dígitos enviados pelo servidor nas respostas para indicar o resultado de uma requisição de forma padronizada. Eles são divididos em famílias informativas: as respostas de sucesso começam com o dígito 2 (como o 200 OK), redirecionamentos começam com 3, e erros do cliente iniciam com 4.

Erros graves do próprio servidor começam com o dígito 5 (como o 500 Internal Server Error). O programador deve configurar o backend para analisar esses códigos retornados pelas APIs externas, pois um erro de digitação de parâmetro resultará em um código 400 Bad Request que precisa ser tratado.

# Classificação e Natureza dos Erros na Rede

O tratamento de erros em sistemas Full Stack exige a compreensão de que falhas em ambientes distribuídos são normais e inevitáveis devido à volatilidade da rede. Os erros podem ser classificados em três categorias principais: falhas de conectividade física, erros de lógica do cliente e indisponibilidade do serviço externo.

Quedas de conexão e tempos de espera esgotados (timeouts) ocorrem na camada física da rede antes mesmo da mensagem chegar ao destino. Erros de lógica envolvem **o envio de dados inválidos ou credenciais expiradas**, enquanto a indisponibilidade acontece quando o servidor remoto sofre uma sobrecarga interna.

# Fluxos de Captura de Exceções com Try-Catch

A técnica básica para evitar que uma falha derrube a execução do servidor backend é o uso de blocos de **controle de exceção** conhecidos como **try-catch**. O código que realiza a chamada HTTP externa é inserido **dentro da cláusula try**, que monitora ativamente qualquer comportamento anômalo ou erro de execução.

Caso a API externa falhe ou a rede caia, o fluxo normal é interrompido imediatamente e o controle é **desviado para o bloco catch**. Dentro do catch, o desenvolvedor insere a lógica para registrar a falha em arquivos de log e estruturar uma resposta amigável para o usuário final, impedindo o travamento do sistema.



# Validação de Dados na Entrada do Backend

Uma linha de defesa crucial contra erros de integração é a validação rigorosa dos dados recebidos do frontend antes que qualquer requisição externa seja disparada. O backend deve checar se os campos obrigatórios estão presentes, se os tipos de dados estão corretos e se as informações respeitam regras de negócio essenciais.

Se um usuário enviar um texto onde deveria constar um valor numérico de ID, o backend deve interceptar o problema imediatamente e retornar um erro. Essa validação prévia poupa recursos de processamento da rede e evita o envio de requisições já reconhecidas como defeituosas para os parceiros externos.

# Estratégias de Repetição Automática (Retry)

Quando uma API externa retorna um erro temporário de rede ou de sobrecarga momentânea, a aplicação pode adotar uma estratégia de repetição automática das tentativas de envio. Em vez de desistir na primeira falha, o backend aguarda uma fração de segundo e dispara a mesma requisição novamente de forma transparente.

Para evitar sobrecarregar ainda mais o serviço remoto, utiliza-se o conceito de **recuo exponencial**, onde o tempo de espera aumenta a cada tentativa falha. Se após três ou quatro tentativas o erro persistir, o sistema interrompe o fluxo e notifica a falha para evitar o travamento de recursos do servidor local.

# O Padrão de Projeto Circuit Breaker

Para proteger o ecossistema de software de falhas em cascata, engenheiros utilizam o padrão de projeto chamado **Disjuntor ou Circuit Breaker**. Esse padrão monitora o volume de erros gerados pelas chamadas para uma determinada API externa ao longo do tempo.

Se a taxa de falhas ultrapassar um limite crítico estabelecido, o disjuntor "abre", **bloqueando temporariamente todas as novas requisições** para aquele serviço quebrado e retornando um erro imediato. Isso evita o desperdício de tempo de processamento com chamadas que sabidamente vão falhar, dando tempo para que o servidor externo se recupere da instabilidade.

# Limitações e Riscos da Dependência de Terceiros

Apesar das vantagens, a dependência crônica de APIs externas introduz riscos de negócio e técnicos que devem ser cuidadosamente gerenciados. A organização perde o controle total sobre a disponibilidade do seu próprio sistema, ficando vulnerável a instabilidades nos servidores de terceiros.

Alterações repentinas na documentação da API, depreciação de funções antigas ou reajustes abusivos nas tabelas de preços podem impactar diretamente o funcionamento e a lucratividade do software local. Além disso, falhas de segurança no provedor externo podem comprometer a privacidade dos dados trafegados pela aplicação.

# Questionário

